

Primary sources & further reading

Part 08 —  Retrieval-Augmented Generation · 08-rag/papers/PAPERS.md · 08-rag/resources/RESOURCES.md



Papers — Retrieval-Augmented Generation

`bash ../_tools/fetch_papers.sh 08-rag` downloads these into this folder.

Read #1 twice. It named the whole field.

#	Paper	Year	Why	Link
1	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — Lewis et al. (Meta AI)	2020	The paper that coined "RAG": a generator conditioned on retrieved passages, trained end-to-end. Nearly every production RAG system is a much simpler, non-differentiable descendant of this idea	arXiv:2005.11401

Dense retrieval and the retriever/reranker split

Paper	Takeaway	Link
Dense Passage Retrieval (DPR) — Karpukhin et al., 2020	Two BERT encoders (query, passage) trained contrastively beat BM25 on open-domain QA — the paper that made dense retrieval credible	arXiv:2004.04906
REALM: Retrieval-Augmented Language Model Pre-Training — Guu et al., 2020	Retrieval integrated into pretraining itself, not bolted on afterward	arXiv:2002.08909
Leveraging Passage Retrieval with Generative Models (Fusion-in-Decoder) — Izacard & Grave, 2020	Encode each retrieved passage separately, fuse in the decoder — scales gracefully to many retrieved passages	arXiv:2007.01282
ColBERT: Efficient and Effective Passage Search via Late Interaction — Khattab & Zaharia, 2020	Token-level late interaction instead of one pooled vector — a middle ground between bi-encoder speed and cross-encoder accuracy	arXiv:2004.12832
ColBERTv2 — Santhanam et al., 2022	Compression + distillation that makes late interaction practical at production scale	arXiv:2112.01488
Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods — Cormack, Clarke & Buettcher, 2009	The RRF paper implemented in <code>code/hybrid_search_and_rerank.py</code> — rank-only fusion, no score calibration needed	PDF (Waterloo)

Query enhancement and iterative retrieval

Paper	Takeaway	Link
Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE) — Gao et al., 2022	Embed a hypothetical LLM-generated answer instead of the raw query — closes the query/document stylistic gap	arXiv:2212.10496
Query Rewriting for Retrieval-Augmented Large Language Models — Ma et al., 2023	Trains a small rewriter model specifically to help downstream retrieval, rather than hand-tuning prompts	arXiv:2305.14283
Interleaving Retrieval with Chain-of-Thought Reasoning (IRCoT) — Trivedi et al., 2022	Retrieve, reason a step, retrieve again — the paper behind the "agentic/iterative RAG" pattern for multi-hop questions	arXiv:2212.10509
Active Retrieval Augmented Generation (FLARE) — Jiang et al., 2023	Decide <i>when</i> to retrieve mid-generation by watching the model's own confidence, instead of retrieving once upfront	arXiv:2305.06983
Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection — Asai et al., 2023	Trains the model to emit special tokens deciding whether to retrieve and whether its own output is well-supported	arXiv:2310.11511

Structured and graph-based retrieval

Paper	Takeaway	Link
From Local to Global: A Graph RAG Approach to Query-Focused Summarization (GraphRAG) — Edge et al. (Microsoft Research), 2024	Extract an entity graph, summarize it hierarchically, and answer global/multi-hop questions from community summaries instead of raw chunks	arXiv:2404.16130

Context behavior and evaluation

Paper	Takeaway	Link
Lost in the Middle: How Language Models Use Long Contexts — Liu et al., 2023	Retrieval accuracy degrades measurably for facts placed in the middle of a long context, even when the model can technically "see" them — the paper behind the production failure mode in <code>notes/02</code>	arXiv:2307.03172
RAGAS: Automated Evaluation of Retrieval Augmented Generation — Es et al., 2023	Reference-free faithfulness, answer relevance, and context precision/recall metrics computed with an LLM judge — the paper <code>code/rag_eval_metrics.py</code> 's faithfulness check is a cheap offline proxy for	arXiv:2309.15217

Foundational IR (pre-dates deep learning, still the vocabulary everyone uses)

Paper	Takeaway	Link
The Probabilistic Relevance Framework: BM25 and Beyond — Robertson & Zaragoza, 2009	The definitive BM25 reference; read this before trusting any BM25 implementation, including the one in this module	Foundations & Trends in IR (PDF)
Cumulated Gain-Based Evaluation of IR Techniques — Järvelin & Kekäläinen, 2002	The original NDCG paper — read it once so <code>code/rag_eval_metrics.py</code> 's formula stops looking arbitrary	ACM DL

The best explainers

- **Pinecone's Learning Center** (vector search, HNSW, hybrid search, chunking) — <https://www.pinecone.io/learn/>
- **LlamaIndex — Building Production-Ready RAG Applications** — https://docs.llamaindex.ai/en/stable/optimizing/production_rag/
- **LangChain — RAG conceptual guide** — <https://python.langchain.com/docs/concepts/rag/>
- **Microsoft Research GraphRAG project page** (with the accompanying blog explainer) — <https://microsoft.github.io/graphrag/>

Resources — Retrieval-Augmented Generation

Start here

Resource	Why	Link
Pinecone Learning Center	The best free running explainer on vector search, HNSW/IVF, hybrid search, and chunking — vendor-run but not vendor-biased in its explanations	https://www.pinecone.io/learn/
LlamaIndex — Building Production-Ready RAG Applications	The most complete practitioner walkthrough of the failure modes this module covers, with working code	https://docs.llamaindex.ai/en/stable/optimizing/production_rag/
LangChain RAG conceptual guide	Clear diagrams of the retrieve-then-generate pipeline and its variants (multi-query, parent-document, self-querying)	https://python.langchain.com/docs/concepts/rag/
Microsoft GraphRAG project page	The reference implementation and accompanying explainer for GraphRAG, straight from the paper's authors	https://microsoft.github.io/graphrag/
MTEB Leaderboard	Compare embedding models on real retrieval benchmarks before picking one — read this before, not after, choosing	https://huggingface.co/spaces/mteb/leaderboard
DeepLearning.AI — Building and Evaluating Advanced RAG (short course, free)	Hands-on notebook covering sentence-window and auto-merging retrieval plus RAGAS-style eval	https://www.deeplearning.ai/sHORT-courses/building-evaluating-advanced-rag/

Vector databases — pick on your actual constraints, not hype

Databa se	What it's for	Link
Chroma	Embedded, zero-ops, the fastest path from "I have some text" to "I can query it" — great for prototyping and small-to-medium corpora	https://github.com/chroma-core/chroma
Qdrant	Production-grade, Rust-based, strong filtered-HNSW support (the filtering strategy actually matters — see notes/01 §4)	https://github.com/qdrant/qdrant
Weaviat e	Production-grade, built-in hybrid search (BM25 + vector) out of the box, GraphQL API	https://github.com/weaviate/weaviate
pgvecto r	A Postgres extension — the right choice when you already run Postgres and don't want a new piece of infrastructure to operate	https://github.com/pgvector/pgvector
Milvus	Built for very large scale (billions of vectors), more operational overhead than the others	https://github.com/milvus-io/milvus
FAISS (Meta)	Not a database — a library of ANN index implementations (HNSW, IVF, IVF-PQ). What several of the databases above use under the hood	https://github.com/facebookresearch/faiss

Retrieval / RAG frameworks and libraries

Repo	What's inside
https://github.com/run-llama/llama_index	The most RAG-specific of the major frameworks — chunking strategies, retrievers, rerankers, query engines all in one place
https://github.com/langchain-ai/langchain	Broader agent/chain framework with a full retrieval module; <code>langchain-community</code> has integrations for every vector DB above
https://github.com/UKPLab/sentence-transformers	Bi-encoder and cross-encoder models, including ready-to-use rerankers — the library behind most of <code>code/hybrid_search_and_rerank.py</code> 's real-world equivalent
https://github.com/stanford-futuredata/ColBERT	Reference implementation of late-interaction retrieval
https://github.com/exploding-gradients/ragas	The reference RAGAS implementation — run it for real (LLM judge, not the offline proxy in this module's code) once you have API access
https://github.com/microsoft/graphrag	Microsoft's reference GraphRAG implementation, indexing pipeline included
https://github.com/xhluca/bm25s	A fast, dependency-light BM25 implementation in Python if you want a real one instead of the from-scratch version in this module

Clone the starter set with: `bash ../_tools/clone_repos.sh 08-rag`

Interactive tools

- **Vespa's ANN visualizer** / **blog series on HNSW** — concrete, implementation-level detail on the graph structure — <https://blog.vespa.ai/>
- **LangSmith** / **LangFuse** — tracing what a RAG pipeline actually retrieved and generated per request, essential once you're past a demo — <https://www.langchain.com/langsmith> · <https://langfuse.com/>

- **Tiktokenizer** — check how much of your context budget a retrieved chunk actually costs — <https://tiktokenizer.vercel.app/>

Communities

- **r/LangChain** and **r/LocalLLaMA** (Reddit) — active, opinionated, and quick to call out RAG anti-patterns
- **LlamaIndex Discord** — <https://discord.gg/dGwcsnxhU>
- **Qdrant Discord** — <https://qdrant.to/discord>
- **MTEB / Hugging Face forums** — for embedding-model-specific questions — <https://discuss.huggingface.co/>

Datasets for building your own retrieval eval set

- **BEIR** (a heterogeneous IR benchmark — good source of realistic query/relevance-judgment triples) — <https://github.com/beir-cellar/beir>
- **MS MARCO** (large-scale passage ranking, the dataset most dense retrievers are benchmarked on) — <https://microsoft.github.io/msmarco/>
- **Natural Questions** (real Google search queries with Wikipedia-grounded answers) — <https://ai.google.com/research/NaturalQuestions>